Notes of Scala and Spark[1]

Yue Wu

June 10, 2026

[1] Based on IEOR4526 ANALYTICS ON THE CLOUD at Columbia University, taught by Prof. Hardeep Johar

Outline

Outline

Cloud Computing

- Distributed File Systems: Data is physically split up and resides in different disks on a cluster
- Distributed programming: programs that run on a system that is located across multiple components (CPUs, RAMs, Disks)
- Functional programming

Imperative vs Functional programming

Functional Programming

- **Pure functions** always produce the same output for the same input and have no side effects. (stateless & referential transparency)
  - **Stateless functions**: functions that do not retain any state information and have no side effects
    - so when we separate the goals, their sequence is not important
  - **Referential transparency**: a function call can be replaced by its result without changing the behavior of the program.
    - i.e., for a given input object, the function must always return the same value
  - **Immutable**: Pure functions must use immutable data.

Functional programs are written as a series of pure function calls.

Because functional programs avoid shared mutable state and side effects, they are easier to parallelize or distribute.

Outline

Outline

Spark

- A framework for cluster computing
- Written in Scala
- A platform, which has APIs for Scala, Python, Java, and R. Objects created using the API are in the calling language
  - Scala API creates Scala objects
  - Python API creates Python objects
- intrinsically lazy. Nothing is evaluated unless there is an action step

Resilient Distributed Datasets(RDD)

A Resilient Distributed Dataset (RDD) is the primary data abstraction used in Spark.

Properties of RDDs:

- immutable(read-only)
- resilient(fault tolerant)
- lazy evaluation
- partitioning and parallel computation

RDD operations:

- transformations: operations that produces a new RDD from an existing RDD
- actions: operations that return actual values (not RDDs) from an RDD

RDD Transformations

- Operations that produces a new RDD from an existing RDD
  - take an RDD as input and produce an RDD as output
  - the set of transformations associated with an RDD is referred to as its **lineage**
  - think of transformations as recipes, steps that will be done when an action is required

- Examples
  - map and flatmap
  - reduceByKey: works on (key,value) pairs, applies a reduce function on each key independently
  - groupByKey:
  - aggregateByKey:
  - combineByKey:
  - foldByKey

RDD Transformations

Table: Comparison between different `byKey` methods

Property	**reduceByKey** & **foldByKey**	**groupByKey**	**aggregateByKey** & **combineByKey**
Shuffle	local aggregates, then shuffles	shuffles raw data(expensive)	optimize local aggregations, then shuffle
Memory	Values are combined using an associative or commutative function	Memory hog	Provides control over memory usage at partition level
Flexibility	Best for commutative and associative functions	Best when finding multiple ad-hoc aggregate functions on grouped data	Best when defining custom aggregate functions

RDD Actions

- actions: operations that return actual values (not RDDs) from an RDD
  - .toDebugString: returns the lineage of an RDD
  - .count: counts the number of elements in the RDD
  - .first: returns the first element of the RDD
  - .take(n): returns the first n elements in an RDD
  - .takeOrdered(n)(function): returns the first n according to the ordering function
  - .collect: collects all the data resulting from the series of operations in one node. Use carefully, because the data must fit in the master node memory!
  - .foreach: applies a function to each element in an RDD
  - reduce
  - countByKey: works on (key,value) pairs, returns the count for each key

Partitioning

- Spark automatically partitions data.
- Size of partition?
  - If using hadoop, Spark uses the HDFS partition size for each partition (default = 512MB). Otherwise, the default is usually = the number of cores on the machine
  - But, partitioning is controllable by the user
- The importance of partitioning?
  - parallel data processing: the data is distributed across nodes with each node simultaneously working on its chunk of the data
  - data locality: partitioning helps keep computation and data together
    - example, if all the records of a key are in the same partition, all computation on the key can be done locally without having to do any network data transfers, which also minimizes the need for shuffles and joins (if computation is by key)
  - scalability: as the data grows, the application can scale with more nodes being added and without affecting the existing partitions
  - fault tolerance: if a node fails, only that part of the data needs to be regenerated

How to Partition

- Goal
  - **Load balancing**: each worker should get an equal share of the work
  - Efficient and suitable for following transformations
- Three partitioning methods for paired data RDDs
  1) Hash Partitioning: Creates key, value pairs by hashing on the key
     - Example: Hash partition **by store** the sales RDD(date, store, units, price) for **per-store** revenue analytics
  2) Range Partitioning: Sequentially allocates data across the partitions
     - Divide the min-max difference by the number of partitions set partition boundaries using the key range sizes
     - Example: preferred for ordered/sequential data, like (date,price) pairs
  3) Custom Partitioning

How to Partition

- Salting: a technique for redistributing more frequent keys to improve overall resource utilization
  - Problem
    - if one or two keys are far more frequent(**skewed**), the machine working on that will work longer while the other machines will be idle
  - Procedure
    - map each key to (key,random_integer)
    - run reducebykey on the new key (key,random_integer)
    - remove the random integer
    - run reducebykey again
  - Useful when
    - the keys are highly imbalanced
    - joining two rdds and the keys in the larger rdd are skewed
- Repartitioning
- Coalescing

Serialization

- Why we need serialization
  -

Converting an object into a format that can be stored or transmitted and then either reconstructed later or reconstructed in another application. Why do objects need to be serialized before being sent to executors in Apache Spark? Because all necessary resources for performing an operation on data must be available on each executor. Because all resources must be available at the executor for an operation to work.

Broadcasting

- How
  - Data or objects are broadcast to ALL executors
  - Executors maintain a copy of the broadcasted object locally
  - The item being broadcast needs to be immutable
- When to broadcast
  - we have a large data object that needs a small data object to complete a task
  - a small data object (e.g., global variables, configuration data) are used repeatedly by executors
  - an ML model is used for further analysis so broadcast the model

Outline

Streaming

- Examples:
  - Processing and displaying live market data
  - Real time A/B testing
  - Figuring out frauds in real time
- Two important time
  - Event time
  - Processing time
- Types of Stream Processing Models
  - Record-at-a-time: Each piece of arriving data is processed as soon as it arrives
  - Micro-batching(Spark): Data is accumulated into small batches that are then processed together

Streaming

Windowing is a stream processing pattern where an incoming data stream is divided into chunks based on temporal boundaries.

Note that arriving data is batched and a window may consist of many batches. Example: gather data every second(one batch) and process the data in five minute chunks(a window, 300 batches).

- Windows
  - fixed windows
    - defined by a "window length"
    - each data point is processed exactly once in its window
    - Example:
  - sliding windows
    - defined by a "slide factor" and a "window length"
    - a data point may get processed multiple times
    - Example:
  - session windows
- Batch sizes
  - if too big, high latency
  - if too small, overhead of processing the stream can result in unpredictable effects on latency

Streaming abstractions in Spark

- RDD based streaming
- Dataframe based streaming, or Structured Streaming

Outline